МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ
федеральное государственное автономное образовательное учреждение
высшего образования «Санкт-Петербургский политехнический
университет Петра Великого»

Институт компьютерных наук и кибербезопасности

Высшая школа технологий искусственного интеллекта

Направление: 02.03.01 «Математика и компьютерные науки»

Google Form Quiz Solver Bot

Архитектура

Янковский Э., Мелещенко С., Гордиенко Ю., Волнухина В.

Санкт-Петербург, 2026

Содержание

1 Требования к технологическому стеку **3**

2 Компонентная диаграмма **4**

2.1 Сервер 4
2.2 Telegram Bot 4
2.3 Пользователь (User) 5
2.4 Администратор (Admin) 5
2.5 Модуль обработки запросов от пользователей (MessageHandlerModule) 5
2.6 Модуль обработки форм (FormSolvingModule) 6
2.7 Модуль LLM-сервиса (LLMSolverModule) 6
2.8 Модуль истории (HistoryModule) 6
2.9 Модуль статуса (RequestStatusModule) 7
2.10 База данных 7
2.11 Kafka 8
2.12 Модуль администрирования (AdminAuthModule) 9
2.13 Модуль Healthcheck (HealthCheckModule) 9
2.14 Модуль списка пользователей (ListUsersModule) 9
2.15 API GigaChat 9
2.16 API Google Forms 10

Приложение. JSON-schema базы данных **11**

1 Требования к технологическому стеку

Язык программирования: Java 25.

Фреймворк: Spring 7.0.6 (WebFlux, Data MongoDB Reactive 2026.0.0, Modulith 2.1.0).

Telegram API: Java Telegram Bot API 9.6.0.

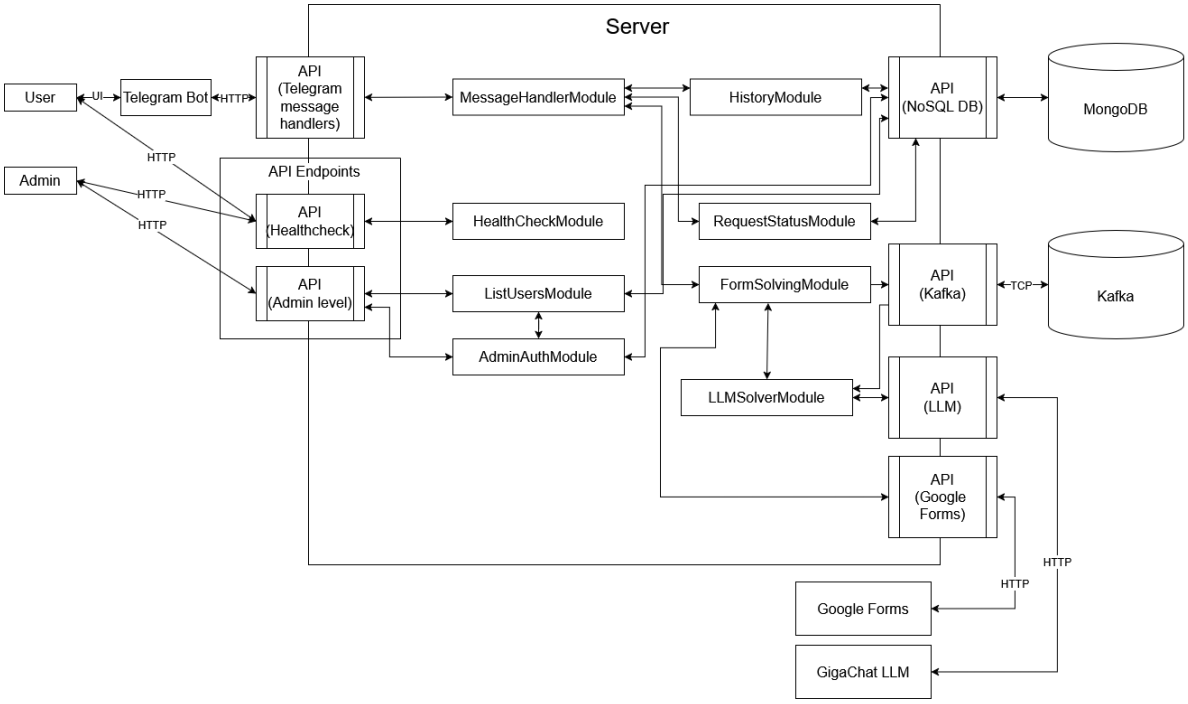
Брокер сообщений: Spring Kafka 4.0.4, Kafka Client 3.9.2.

База данных: MongoDB Driver 5.4.0.

Контейнеризация: Docker 29.1.x, Docker Compose 5.0.2.

Тестирование: JUnit 5.12.0, Testcontainers 2.0.2.

2 Компонентная диаграмма



2.1 Сервер

Роль: Центральный компонент системы.
Функционал:

- Принимает запросы от Telegram-бота и запросы по с endpoint-ов.
- Координирует взаимодействие между Telegram-ботом и другими компонентами системы.

2.2 Telegram Bot

Роль: Пользовательский интерфейс.
Функционал:

- Принимает сообщения и запросы от пользователей через Telegram.
- Передает сообщения на сервер.

2.3 Пользователь (User)

Роль: конечный потребитель сервиса.
Функционал:

- Взаимодействие с системой через Telegram-бота.
- Может выполнить проверку состояния системы через endpoint /healthcheck (без верификации).

2.4 Администратор (Admin)

Роль: пользователь с увеличенными правами.
Функционал:

- Осуществляет административные взаимодействия с HTTP API.
- Имеет доступ к списку всех пользователей.
- Получает доступ через endpoint /auth с данными логина и пароля.

2.5 Модуль обработки запросов от пользователей (MessageHandlerModule)

Роль: Разбор команд полученных через API и распределение задач на другие модули.
Функционал:

- Отдает запрос в модуль для решения формы по url.
- Отдает запрос на статус запроса пользователя по userId и requestId.
- Отдает запрос на сохранение истории, а также на ее получение по userId и formId.
- Обрабатывает ответы от модулей и формирует ответ для внешнего API бота.

2.6 Модуль обработки форм (FormSolvingModule)

Роль: Обработка форм, подготовка данных к передачи LLM.
Функционал:

- По полученной из другого модуля ссылке проверяет подходит ли она под формат обработки, а также получает информацию о форме; возвращает результат.
- Получает структуру формы: список вопросов, тип вопросов, их содержание.
- Добавляет задачи в очередь на выполнение.
- Приводит данные к однородному виду, передает их модулю запросов к LLM по требованию.

2.7 Модуль LLM-сервиса (LLMSolverModule)

Роль: Генерация ответов на вопросы формы с помощью LLM.
Функционал:

- Получает задачи из очереди на выполнение.
- Запрашивает описание формы, вопросов, а также их содержание от другого модуля.
- Формирует промпт для всей формы.
- Обращается к API LLM.
- Возвращает список ответов с уверенностью.

2.8 Модуль истории (HistoryModule)

Роль: Хранение и предоставление истории решений.
Функционал:

- Сохраняет каждый запрос (`requestId`, `userId`, статус, дата, название формы) в MongoDB.

- При запросе из другого модуля возвращает список запросов для конкретного пользователя за заданный период.
- Обеспечивает сохранение форм с ответами.

2.9 Модуль статуса (RequestStatusModule)

Роль: Проверка текущего состояния обработки.
Функционал:

- При запросе из другого модуля возвращает статуст запроса для заданного пользователя и запроса.
- Возвращает статус: `PENDING` (в очереди), `PROCESSING` (обрабатывается), `COMPLETED` (успех), `FAILED` (ошибка).

2.10 База данных

Роль: Хранение всех данных системы.
Функционал:

- Хранение пользователей системы.
- Хранение решенных форм для каждого пользователя.
- Хранение истории для каждого пользователя.
- Хранение статусов запросов.
- Хранение зашифрованных данных логинов и паролей администраторов для API.

Коллекции:

- `AdminData` – login; passwdSHA.
- `Users` – chatId; userId; name; hasCurrentRequest;
  savedForms:
  - formId;

  history:

  - formId,
  - status,
  - solvedDate.

- `Forms` – userId (связь с Users); formId; formName; isSolved; questions:
  - type,
  - body,
  - answer.

В приложении приведена JSON-schema для хранящихся данных.

На рисунке 1 изображена схема базы данных.

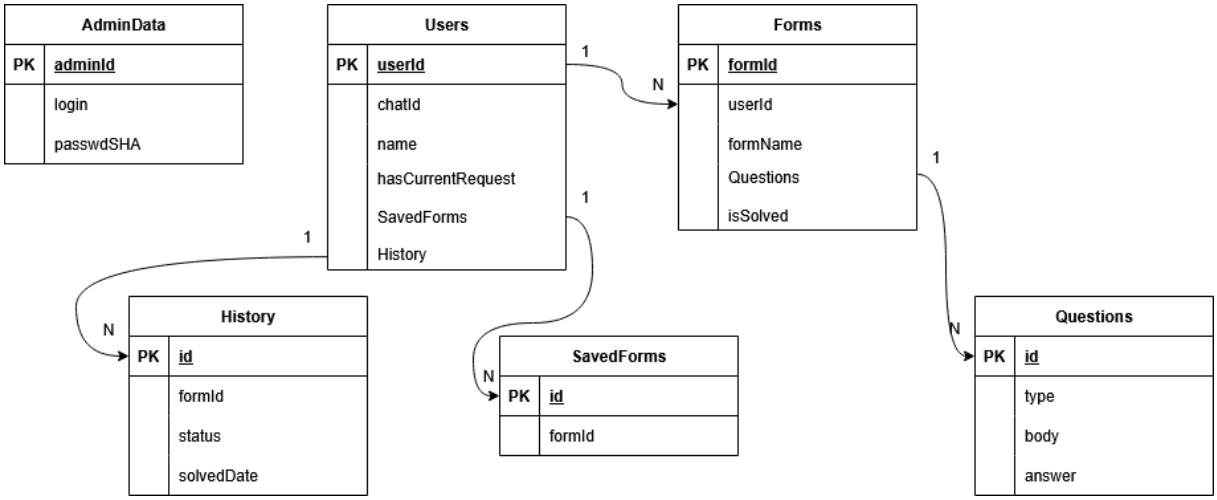


Рис. 1. Схема базы данных

2.11 Kafka

Роль: Асинхронная передача задач на обработку LLM.
Функционал:

- Топик `quiz.solve.requests` – сообщения с `requestId`.
- Producer (в FormSolvingModule) отправляет сообщения.
- Consumer (LLMSolverModule) читает и запускает обработку.

2.12 Модуль администрирования (AdminAuthModule)

Роль: Поддержка аунтификации администраторов через API.
Функционал:

- Обрабатывает запросы к endpoint-у /auth.
- Генерирует API ключи при правильных данных переданных на /auth.
- При запросе из внешних модулей верифицирует ключи API.

2.13 Модуль Healthcheck (HealthCheckModule)

Роль: Системный мониторинг.
Функционал:

- Реализует проверку работоспособности системы (endpoint /healthcheck).
- Доступен без аунтификации по ключу API.
- Предоставляет информацию о сервере, а также о комнаде разработчиков.

2.14 Модуль списка пользователей (ListUsersModule)

Роль: получение списка пользователей.
Функционал:

- Обрабатывет запросы на endpoint /users.
- Получает список всех пользователей из базы данных и возвращает их администратору.
- обращается к другому модулю за верификацией ключа API.

2.15 API GigaChat

Роль: Внешний LLM-провайдер для генерации ответов.
Функционал:

- Принимает POST-запросы с промптами.
- Возвращает текст ответа и (при запросе) уверенность в процентах.

2.16 API Google Forms

Роль: Чтение структуры формы.
Функционал:

- Позволяет получить метаданные формы (заголовок, описание, список вопросов с типами и вариантами).
- Используется сервисным аккаунтом Google (JSON-ключ).

Приложение. JSON-schema базы данных

```
1  {
2    "$schema": "http://json-schema.org/draft-07/schema#",
3    "title": "Google Forms solver bot database schema",
4    "type": "object",
5    "properties": {
6      "AdminData": {
7        "type": "object",
8        "properties": {
9          "login": {
10           "type": "string",
11           "description": "Admin login"
12         },
13         "passSHA": {
14           "type": "string",
15           "description": "Password hashed with SHA algorithm"
16         }
17       },
18       "required": ["login", "passSHA"],
19       "additionalProperties": false
20     },
21     "Users": {
22       "type": "array",
23       "items": {
24         "type": "object",
25         "properties": {
26           "chatId": {
27             "type": "string",
28             "description": "Chat identifier from telegram"
29           },
30           "userId": {
31             "type": "integer",
32             "description": "User ID for internal use"
33           },
34           "name": {
35             "type": "string"
36           },
37           "hasCurrentRequest": {
38             "type": "boolean"
39           },
40           "savedForms": {
41             "type": "array",
42             "items": {
43               "type": "object",
44               "properties": {
45                 "formId": {
46                   "type": "string"
```

```
                }
              },
              "required": ["formId"],
              "additionalProperties": false
            },
            "description": "List of formIds for forms that are saved for that user"
          },
          "history": {
            "type": "array",
            "items": {
              "type": "object",
              "properties": {
                "formId": {
                  "type": "integer"
                },
                "status": {
                  "type": "string"
                },
                "solvedDate": {
                  "type": "string",
                  "format": "date-time",
                  "description": "Date when form was solved"
                }
              },
              "required": ["formId", "status", "solvedDate"],
              "additionalProperties": false
            }
          }
        },
        "required": ["chatId", "userId", "hasCurrentRequest"],
        "additionalProperties": false
      }
    },
    "Forms": {
      "type": "array",
      "items": {
        "type": "object",
        "properties": {
          "ownerId": {
            "type": "integer",
            "description": "Reference to userId"
          },
          "formId": {
            "type": "integer"
          },
          "formName": {
```

```
            "type": "string"
          },
          "Questions": {
            "type": "array",
            "items": {
              "type": "object",
              "properties": {
                "type": {
                  "type": "string",
                  "description": "Question type, e.g. text, number, choice"
                },
                "body": {
                  "type": "string",
                  "description": "Question text"
                },
                "answer": {
                  "type": ["string", "number", "array", "null"],
                  "description": "Answer provided (type varies)"
                }
              },
              "required": ["type", "body"],
              "additionalProperties": false
            }
          },
          "isSolved": {
            "type": "boolean"
          }
        },
        "required": ["owner", "formId", "Questions", "isSolved"],
        "additionalProperties": false
      }
    }
  },
  "required": ["AdminData", "Users", "Forms"],
  "additionalProperties": false
}
```